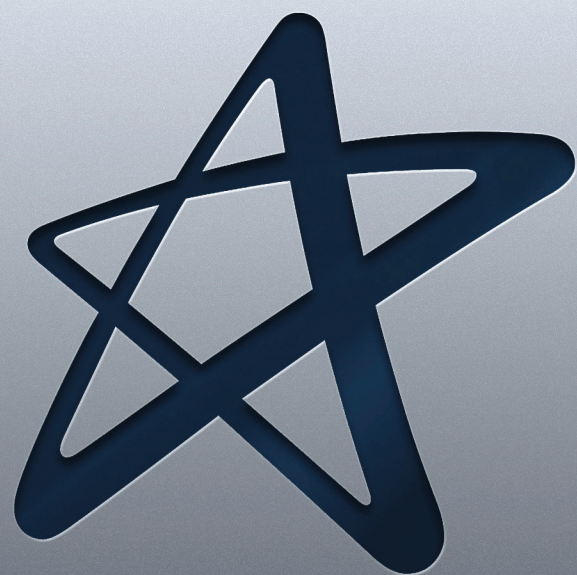


# **Compiladores e Interpretadores**



**Cruzeiro do Sul Virtual**  
Educação a distância



# Material Teórico



## Análise Semântica

### **Responsável pelo Conteúdo:**

Prof. Dr. Cleber Silva Ferreira da Luz

### **Revisão Textual:**

Prof.<sup>a</sup> Dr.<sup>a</sup> Luciene Oliveira da Costa Granadeiro



# UNIDADE

## Análise Semântica



- Introdução;
- Análise Semântica;
- Considerações Finais.



### OBJETIVOS DE APRENDIZADO

- Apresentar todos os conceitos do analisador semântico;
- Demonstrar na prática como é criado um analisador semântico.







# Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



## Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

## Introdução

Vimos que a fase de análise de uma compilação corresponde a 3 análises, análise léxica, sintática e a semântica. A análise léxica é responsável por percorrer o código fonte, detectando padrões e obtendo os *tokens*. Já a análise sintática é responsável por verificar as estruturas das instruções do código fonte. Em suma, o analisador léxico obtém os *tokens* e envia para o analisador sintático verificar a ordem dos mesmos, isto é, verifica se as regras sintáticas da linguagem estão sendo obedecidas. Nesta unidade, vamos estudar a análise semântica. Nessa análise, são realizadas análises que não são possíveis de realizar pelo analisador sintático e muito menos pelo analisador léxico. A análise semântica é responsável por verificar aspectos relacionados ao significado das instruções, se realmente aquela instrução faz sentido. Na análise semântica, são realizadas análises como:

- Verificação de tipo de variável com o valor atribuído para ela, por exemplo, não é possível colocar uma *string* em uma variável do tipo inteiro;
- Verificação da declaração da variável antes do uso, por exemplo, não é possível usar uma variável sem antes a declarar;
- Verificação sobre o escopo da variável, isto é, verificação se a variável esta sendo utilizada onde ela pode ser usada (escopo);
- Verificação sobre os identificadores, por exemplo, o identificador x (um conjunto de caracteres) é uma variável ou uma função?

Perceba que essas verificações não são possíveis de serem verificadas por autômatos e expressões regulares. A análise semântica é explorada com mais detalhes na próxima seção.

## Análise Semântica

O processo de compilação segue um fluxo. Primeiro, é realizada a análise léxica, que obtém os *tokens* e que são passados para a próxima etapa, a análise sintática. A análise sintática, por sua vez, obtém os *tokens* e os deixam em uma estrutura de dados onde é mais fácil visualizar a hierarquia entre eles, essa estrutura de dados é dada por uma árvore sintática, também chamada de árvore de derivação, ou simplesmente árvore. O analisador sintático cria a árvore de derivação e a envia para a próxima etapa, a análise semântica. O analisador semântico recebe a saída do analisador sintático (a árvore de derivação) para realizar a análise semântica. O fluxo da fase de análise é ilustrado na Figura 1.



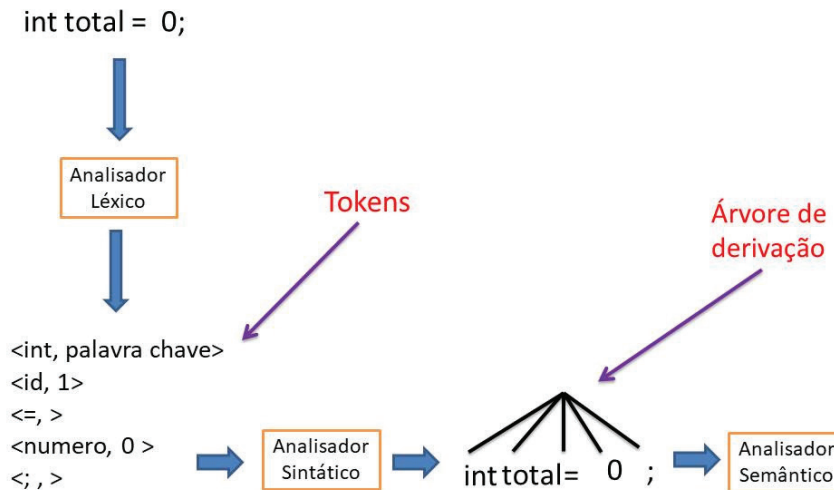


Figura 1 – Fluxo da fase de análise

Fonte: Acervo do conteudista

Na análise semântica, são realizadas análises como verificação de compatibilidade de tipo de variável, verificação do escopo das variáveis, verificação da correspondência entre parâmetros e a verificação das declarações de variáveis. É importante observar que a análise léxica também faz uso da **tabela de símbolos** da linguagem. A análise semântica é realizada por um analisador semântico que, além de realizar a análise no código fonte, deve reportar erros semânticos para o usuário. Por exemplo, considere a seguinte instrução `count = count + 1`. Nesse caso, o analisador semântico realiza as seguintes análises:

- O identificador `count` foi declarado? Caso negativo, erro semântico;
- O identificador `count` é uma variável? Caso negativo, erro semântico;
- Qual o escopo da declaração da variável `count`, local ou global? Se a variável for local e tentar acessá-la em outro método, por exemplo, erro semântico;
- Qual é o tipo da variável `count`? O valor atribuído do lado direito é compatível?

Um analisador semântico possui a capacidade de responder a todas essas perguntas. Vale ressaltar que, expressões regulares, autômatas e uma Gramática Livre de Contexto (GLC) não conseguem responder essas perguntas. Todavia, uma GLC pode ser usada como guia na análise semântica de programas. Essa estratégia é denominada como “Tradução dirigida pela sintaxe” (LOUDEN, 2004). Em outras palavras, a tradução dirigida pela sintaxe diz que a semântica de uma linguagem é extremamente ligada com a sintaxe da linguagem em questão.

A análise semântica depende muito da linguagem-alvo. Dessa forma, não há um padrão para descrever um analisador semântico. No geral, um método que é bastante utilizado em uma análise semântica consiste na identificação de atributos, ou propriedades, de entidades da linguagem que precisem ser computadas. Assim, é possível escrever equações de atributos, ou regras semânticas, que expressam como a computação desses atributos se relaciona com as regras gramaticais da linguagem. Esse conjunto de atributos e equações é denominado de gramática de atributos (LOUDEN, 2004).

## Gramática de Atributo

A gramática de atributo é útil para as linguagens que obedecem ao princípio de semântica dirigida pela sintaxe, o qual determina que o conteúdo semântico de um programa deve ser fortemente relacionado com sua sintaxe. Dessa forma, a semântica dirigida pela sintaxe é realizada junto com a análise sintática. Toda vez que uma regra de produção é processada, uma verificação semântica é realizada. Todas as linguagens modernas seguem essa propriedade (LOUDEN, 2004). Toda vez que uma regra sintática é executada, é executada também, uma regra semântica associada.

Habitualmente, as gramáticas de atributos são escritas em forma de tabela, com cada regra gramatical listada junto com o conjunto de equações de atributos, ou regras semânticas associadas.

Tabela 1

| Regra gramatical | Regra semântica                  |
|------------------|----------------------------------|
| Regra 1          | Equações de atributos associados |
| .                | .                                |
| .                | .                                |
| .                | .                                |
| Regra n          | Equações de atributos associados |

Na Gramática de Atributo, para cada regra sintática (gramatical) é expressa uma regra semântica. Quando uma regra sintática é executada, logo, uma regra semântica é executada. Por exemplo, observe a tabela a seguir;

Tabela 2

| Regra Gramatical      | Regra Semântica                                    |
|-----------------------|----------------------------------------------------|
| $A \rightarrow X + Y$ | $A.code ::= X.code \parallel "+" \parallel Y.code$ |

Na Gramática de Atributo, os valores dos atributos são especificados de acordo com as regras semânticas, isto é, é realizada uma verificação do atributo com o seu valor. Por exemplo, podemos verificar se uma variável está armazenando um valor que corresponde ao seu tipo. Para exemplificar, vamos supor que a Y seja uma variável. Então, é definido um campo em Y, que será chamado de *code*, assim, *Y.code* é um campo em Y que irá armazenar o valor Y. Vamos supor, ainda, que a variável Y está armazenando o valor 20, então, o campo *Y.code* irá armazenar o número 20. Em suma, a Gramática de Atributo especifica os valores dos atributos, associando regras semânticas às produções da gramática.

Agora, observe a tabela a seguir:

Tabela 3

| Regra Gramatical      | Regra Semântica                                 |
|-----------------------|-------------------------------------------------|
| $A \rightarrow X + Y$ | $A.val ::= X.val \parallel "+" \parallel Y.val$ |

A regra gramatical acima corresponde à regra sintática de uma atribuição. Nessa atribuição, está sendo somando o valor de  $X + Y$  e atribuído à variável  $A$ . A regra sintática  $A \rightarrow X + Y$  consegue verificar se a estrutura da atribuição está correta ou não. Associada com essa regra sintática está a regra semântica  $A.val ::= X.val \parallel "+" \parallel Y.val$  que permite especificar o valor associado a cada variável, assim, é possível verificar se cada variável contém um valor que corresponde com o seu tipo. Val é chamado de atributo.

Um **atributo é qualquer propriedade** de uma construção de linguagem de programação. Alguns exemplos de atributos:

- Tipo de dados de uma variável;
- Valor de uma expressão;
- Quantidade de dígitos significativos em um número;
- Um atributo é uma variável adicionada aos símbolos de uma gramática. Para armazenar valores.

Os atributos podem ser fixados antes do processo de compilação, ou durante a execução de um programa. O processo de computar um atributo e associar o seu valor recebe o nome de amarração do atributo (LOUDEN, 2004).

A amarração do atributo com o seu valor pode ser realizada em tempos diferentes. Por exemplo, a amarração pode ser realizada estaticamente antes do processo de compilação ou dinamicamente durante a execução do programa.

Para tornar as ações semânticas mais efetivas, podem-se associar variáveis aos símbolos da gramática. Assim, os símbolos gramaticais passam a conter atributos (ou parâmetros) capazes de armazenar valores durante o processo de reconhecimento. Toda vez em que uma regra de produção é usada no processo de reconhecimento de uma sentença, os símbolos gramaticais dessa regra são “alocados” juntamente com seus atributos.

As regras sintáticas realizam derivações que podem ser expressas como uma árvore. Tal árvore é chamada de árvore de derivação. No caso da análise semântica, é criada uma árvore anotada. Uma árvore anotada é uma árvore de derivação com valores em cada nó. Assim, temos que, se  $X$  for um símbolo gramatical, e  $a$  for um atributo de  $X$ , temos uma associação  $X.a$ . O atributo mais significativo de um número é o seu valor, que recebe o nome de val. A regra gramatical  $dígito \rightarrow 0$  indica que o dígito tem o valor 0 nesse caso. Isso pode ser expresso como  $dígito.val = 0$ . Assim, uma árvore anotada é uma árvore de derivação que possui campos para armazenar valores dos atributos.

Em outras palavras, as regras de uma semântica dirigida pela sintaxe são aplicadas na construção de uma árvore de derivação e, então, usadas para avaliar todos os atributos em cada um dos nós da árvore de derivação. Uma árvore de derivação, mostrando os valores de seus atributos é denominada árvore de derivação anotada.

Árvore de derivação anotada para o número 345.

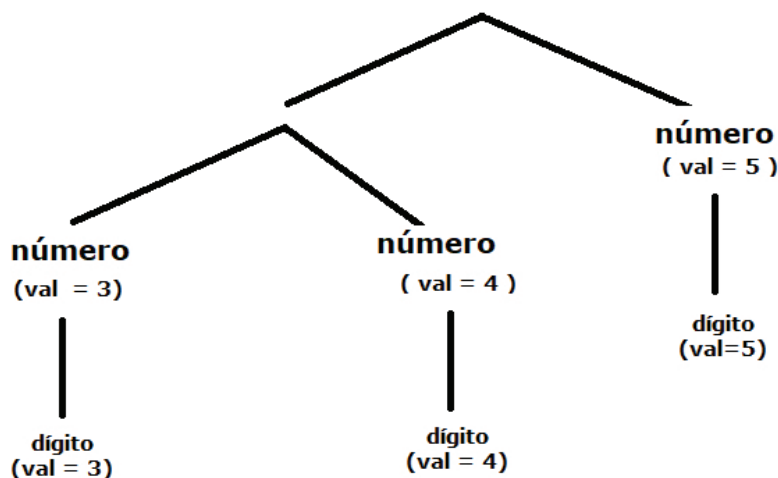


Figura 2 – Árvore Anotada

Fonte: Acervo do conteudista

Após criar a árvore anotada, é preciso percorrê-la e avaliar todos seus atributos. Para isso, veremos como uma árvore pode ser percorrida. Percorrer uma árvore significa visitar todos os seus nós, e para isso há 3 maneiras de visitar os nós de uma árvore. Todavia, antes de entrarmos em percurso de árvore, é necessário entender o que é um atributo sintetizado e o que é um atributo herdado. As próximas seções explicam com detalhes o que é atributo sintetizado e atributo herdado. Começando pelo atributo sintetizado.

### Atributo Sintetizado

Atributo sintetizado somente depende dos atributos de seus filhos. Esse atributo é processado a partir dos atributos dos nós-filhos (abaixo do próprio nó). Uma gramática de atributos em que todos os atributos são sintetizados é denominada gramática S-atribuída (LOUDEN, 2004). Uma gramática S-atribuída pode ser implementada utilizando o algoritmo LR(1).

Em palavras mais simples, para calcular o valor de um atributo sintetizado, primeiro é necessário calcular (ou obter) o valor dos atributos dos filhos. Por exemplo, considere a seguinte regra sintática  $T \rightarrow FG$  que está associada com a seguinte regra semântica  $T.s ::= f(F.f, G.g)$ . Isso significa que, para obter o valor do atributo  $T$  ( $s$ ), é necessário primeiro obter o valor do atributo  $F$  ( $f$ ) e o valor do atributo  $G$  ( $g$ ), só depois de obter o atributo de  $F$  e  $G$  que será possível estimar o valor do atributo  $T$ .

Tabela 4

| Regra Gramatical   | Regra Semântica       |
|--------------------|-----------------------|
| $T \rightarrow FG$ | $T.s ::= f(F.f, G.g)$ |

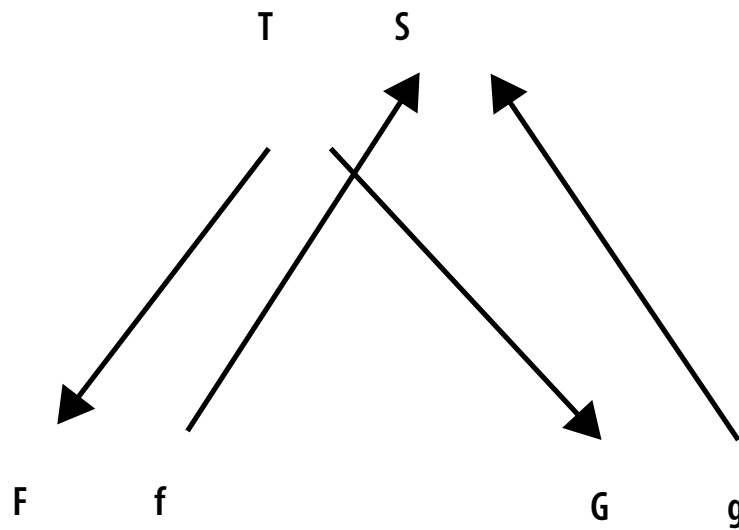


Figura 3 – Atributo Sintetizado

### Atributo Herdado

Atributo herdado depende de, pelo menos, um atributo do pai (ou de um dos irmãos). Observe a tabela a seguir:

Tabela 5

| Regra Gramatical   | Regra Semântica       |
|--------------------|-----------------------|
| $H \rightarrow XY$ | $H.s ::= f(X.x, Y.y)$ |

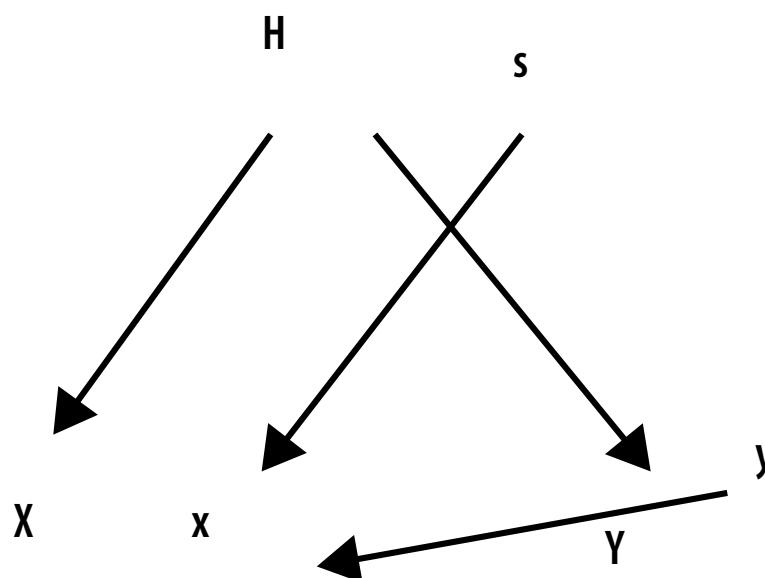


Figura 4 – Atributo Herdado

Atributos herdados são processados a partir de nós-pais ou nós-irmãos (acima ou ao lado do próprio nó). Vale ressaltar que, símbolos terminais podem ter atributos sintetizados, mas não herdados. Os atributos dos terminais são valores léxicos retornados da análise léxica.



## Percurso em Árvore

Basicamente, há três tipos de percursos em árvores que são:

- Pré-ordem;
- Em ordem;
- Pós-ordem.

O percurso pré-ordem visita primeiro a raiz, depois a sub-árvore esquerda e por último a sub-árvore direita. Por exemplo, considere a árvore apresentada a seguir:

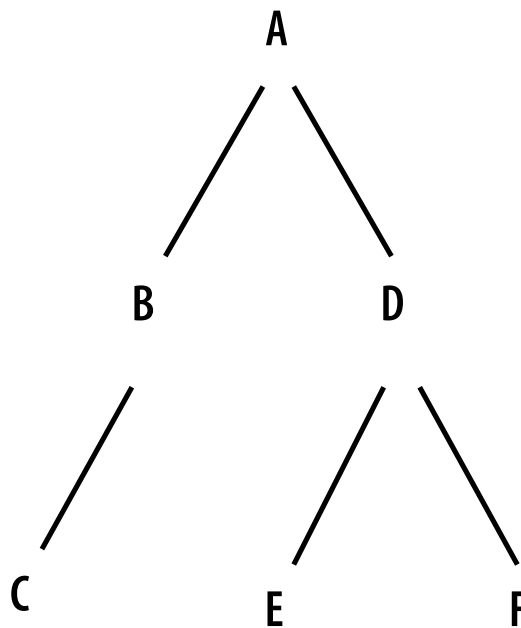


Figura 5 – Árvore para percurso

Em um percurso “pré-ordem”, a ordem dos nós visitados seria: A B C D E F.

Agora, o percurso “em ordem” visita primeiro a sub-árvore esquerda, depois a raiz e, por último, a sub-árvore à direita. Considerando a árvore apresentada na Figura 5, um percurso em ordem iria visitar os vértices C B A E D F.

Já o percurso “pós-ordem”, visita primeiro a sub-árvore esquerda, depois a sub-árvore à direita e, por último, a raiz. Também considerando a árvore apresentada na Figura 5, temos a seguinte ordem de visitação C B E F D A.

## Árvore Anotada

Com certeza, você deve estar se perguntando agora:

- Como construir uma árvore de derivação anotada?
- Qual a ordem de avaliação dos atributos?

Nesta seção, iremos responder a essas duas perguntas. Vamos começar pela primeira pergunta, “Como construir uma árvore de derivação anotada?”. A construção de uma árvore anotada segue a construção de uma árvore de derivação. A árvore anotada é uma árvore de derivação com campos para armazenar os valores dos atributos. Como mencionado anteriormente, toda vez que uma regra sintática é executada, uma regra semântica é executada, formando uma única árvore. A segunda pergunta é “Qual a ordem de avaliação dos atributos?”. Bom, antes de avaliar um atributo de um nó de uma árvore de derivação, devemos avaliar todos os atributos dos quais depende o seu valor. Por exemplo, se todos os atributos forem sintetizados, então precisamos avaliar os atributos val de todos os filhos de um nó antes de avaliar o atributo val do próprio nó. Por exemplo, considerando os **atributos sintetizados**, podemos **avaliar atributos** em qualquer ordem ascendente, por exemplo, por um caminho **pós-ordem** na árvore de derivação.

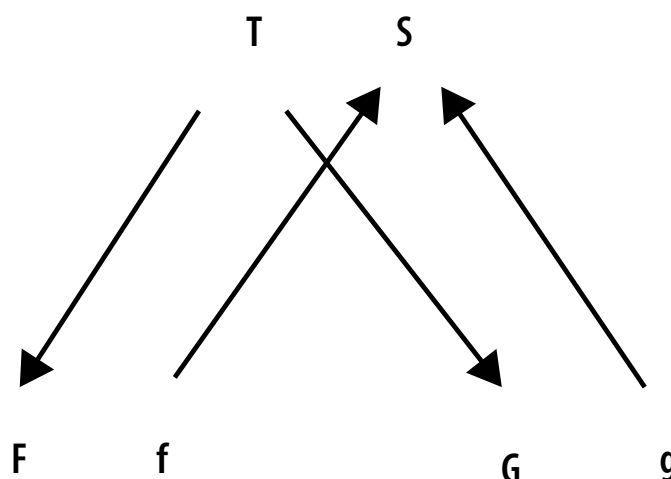


Figura 6 – Atributo Sintetizado

Quando a árvore de derivação é gerada utilizando a Análise Ascendente, árvore anotada é formada avaliando os atributos sintetizados. Vamos criar a árvore anotada para a expressão  $3 * 5 + 4$  utilizando a Análise Ascendente. Para isso, vamos considerar as seguintes regras sintáticas e as seguintes regras semânticas que são apresentadas na tabela a seguir:

Tabela 6

| Regra Gramatical         | Regra Semântica         |
|--------------------------|-------------------------|
| 1) $L \rightarrow E$     | $L.val = E.val$         |
| 2) $E \rightarrow E + T$ | $E.val = E.val + T.val$ |
| 3) $E \rightarrow T$     | $E.val = T.val$         |
| 4) $T \rightarrow T * F$ | $T.val = T.val * F.val$ |
| 5) $T \rightarrow F$     | $T.val = F.val$         |
| 6) $F \rightarrow (E)$   | $F.val = E.val$         |
| 7) $F \rightarrow digit$ | $F.val = digit.lexval$  |

A ideia geral consiste em, toda vez que for executada uma regra sintática, uma regra semântica é executada e o valor do atributo é calculado. Pelo fato de ser uma análise ascendente, devemos começar pelas folhas e chegar à raiz. Assim, temos:

$$\text{digit.lexval} = 3 \quad * \quad \text{digit.lexval} = 5 \quad + \quad \text{digit.lexval} = 4$$

Figura 7 – Folhas

Fonte: Acervo do conteudista

Agora, aplicamos a regra sintática 7 que de digit chegamos a F. Quando executamos uma regra sintática, devemos executar a regra semântica associada a ela, que, neste caso, é  $F.val = \text{digit.lexval}$ . Logo, a nossa árvore fica desta forma:



Figura 8 – Aplicação da regra 7

Fonte: Acervo do conteudista

Aplicando a regra 5 temos:



Figura 9 – Aplicação da regra 5

Fonte: Acervo do conteudista

Aplicando a regra 7, temos:

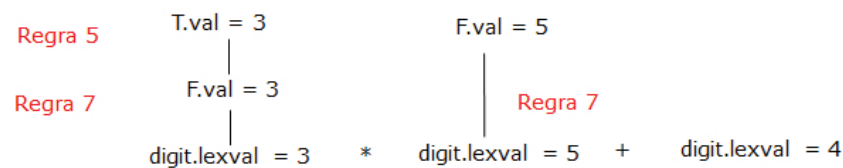


Figura 10 – Aplicação da regra 7

Fonte: Acervo do conteudista

Agora, aplicando a regra 4, temos:

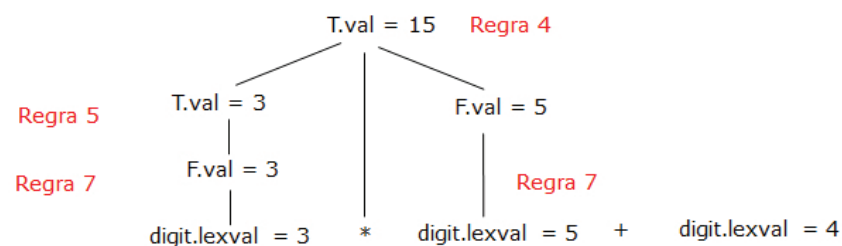


Figura 11 – Aplicação da regra 4

Fonte: Acervo do conteudista

É importante observar que a regra 4 diz que o valor de T.val será o próprio valor de T.val \* por F.val. Assim, o valor de T.val é 15, por  $3 * 5 = 15$ .

Agora, aplicamos a regra 3 e temos:

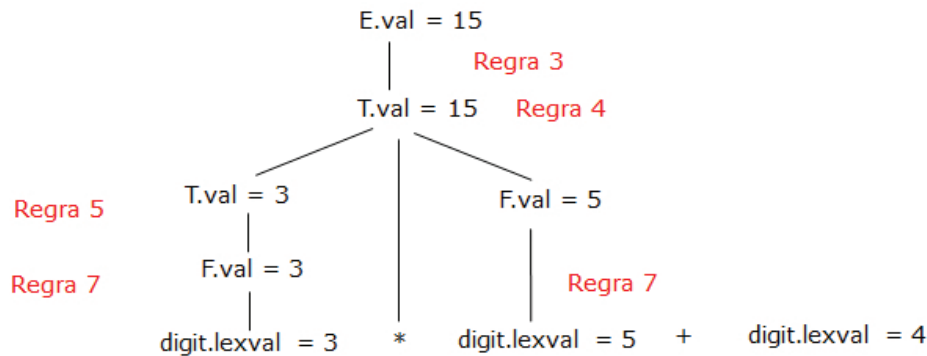


Figura 12 – Aplicação da regra 3

Fonte: Acervo do conteudista

Aplicando a regra 7:

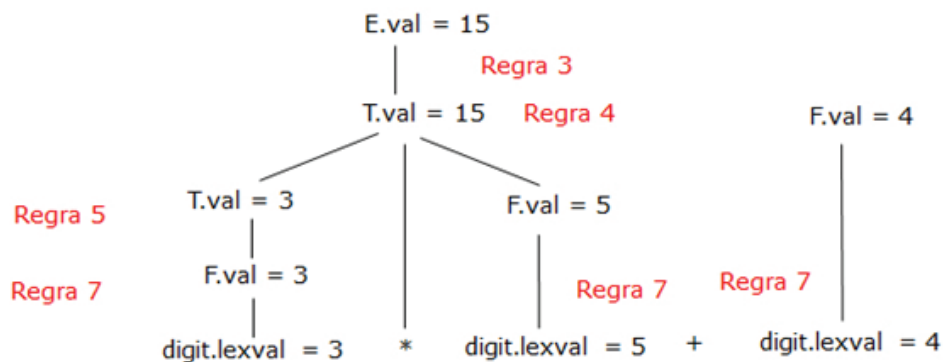


Figura 13 – Aplicação da regra 7

Fonte: Acervo do conteudista

Agora, aplicando a regra 5:

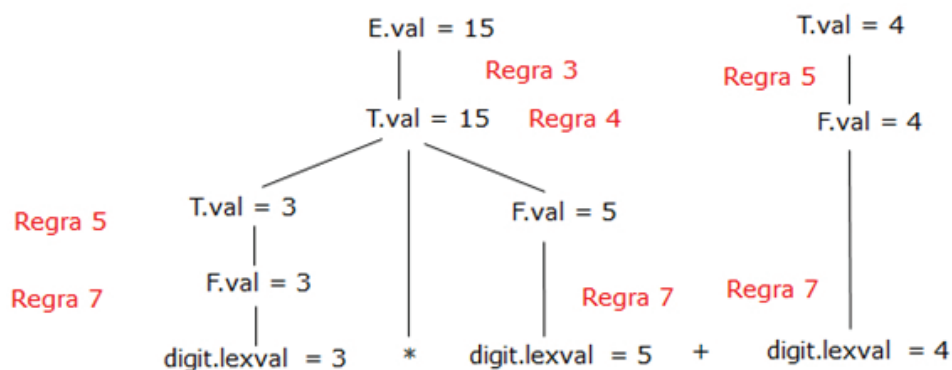


Figura 14 – Aplicação da regra 5

Fonte: Acervo do conteudista

Aplicando a regra 2 temos:

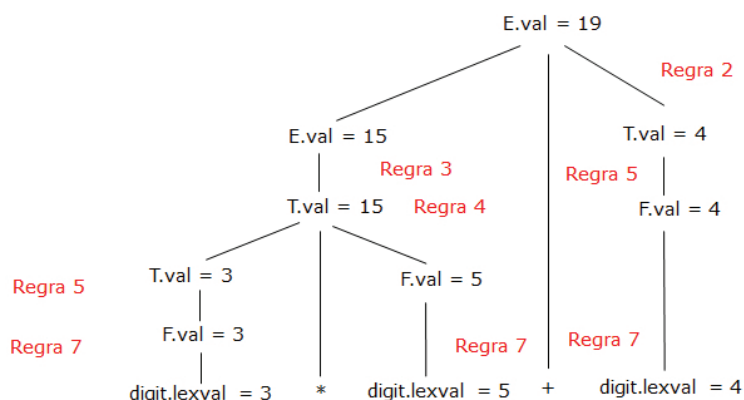


Figura 15 – Aplicação da regra 2

Fonte: Acervo do conteudista

E, por último, aplicamos a regra 1 e chegamos ao topo:

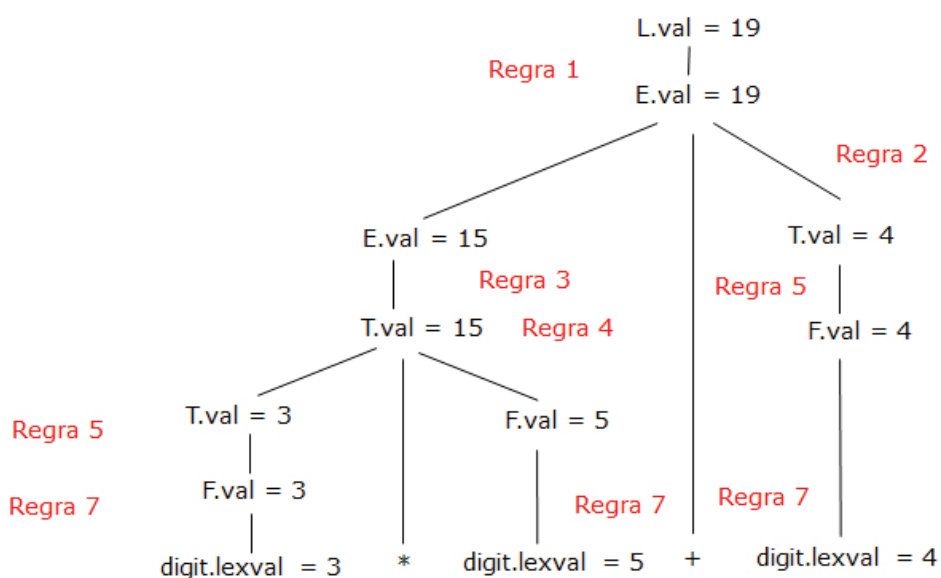


Figura 16 – Aplicação da regra 1

Fonte: Acervo do conteudista

E, finalmente, conseguimos criar a árvore anotada com sucesso. Nessa árvore, devemos observar que, cada um dos nós não terminais possui um atributo `val` calculado em uma ordem ascendente, e chegamos aos valores resultantes associados a cada nó. Exemplo, depois de calcular `T.val = 3` e `F.val = 5`, aplicamos a regra que diz que `T.val` é o produto desses dois valores, ou seja, 15.



## Em Síntese

A análise semântica é a última análise realizada na fase de análise. Nesta fase, são realizadas verificações que as análises léxicas e sintáticas não conseguem realizar, como, por exemplo, verificação de compatibilidade de tipo com o valor associado, uso de variáveis antes do uso, escopo de variáveis, entre outros.

Essa análise é realizada por um analisador semântico que realiza a análise por criar uma árvore anotada. Uma árvore anotada é uma árvore de derivação com campos em cada nó, para que se possa calcular o valor cada nó. A análise semântica utiliza a de “Tradução dirigida pela sintaxe”. Essa estratégia diz que a semântica da linguagem deve estar totalmente ligada com a sintaxe da linguagem.



# Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:



## Livros

### Conceitos de Linguagens de Programação

SEBESTA, R. W. Conceitos de Linguagens de Programação. Rio Grande do Sul: Bookman, 2011.

### Optimizing compilers for modern architectures: a dependence-based approach

KENNEDY, K.; ALLEN, R. – *Optimizing compilers for modern architectures: a dependence-based approach*, Morgan Kaufmann Publishers Inc. San Francisco, CA, USA, 2002.

### Introdução à Compilação

NETO, J. J. Introdução à Compilação. 2. ed. Amsterdã: Elsevier, 2016.



## Leitura

### Compiler Construction

WAITE, W. M.; GOOS, G. *Compiler Construction*, 1996.

<http://bit.ly/331J3Kt>

## Referências

AHO, A. V. **Compiladores: Princípios, Técnicas e Ferramentas**. 2. ed. Pearson. (*e-book*)

APPEL, A. W. **Modern Compiler Implementation in Java**. 2. ed. New York: Cambridge University Press, 2002.

BERGMANN, S. D. **Compiler Design: Theory, Tools, and Examples**, 2016. <<http://www.cs.cmu.edu/~aplatzer/course/Compilers/waitegoos.pdf>>. (*e-book*)

LOUDEN, K. C. **Compiladores: Princípios e Práticas**. São Paulo: Pioneira Thomson Learning, 2004.

PRICE, A. M. A. **Implementação de Linguagens de Programação: Compiladores**. 2. ed. Porto Alegre: Sagra Luzzatto, 2001.



**Cruzeiro do Sul**  
Educatonal